

Akıllı Tarım Yönetim Sistemi

Teknik Araştırma ve Karar Dokümanı

Cihan Akalın - Yazılım Mühendisliği (1. Sınıf)

20 Mart 2026

İçindekiler

1	Mimari Yaklaşımların Karşılaştırmalı Analizi	2
1.1	Mantık 1: Monolitik Mimari	2
1.2	Mantık 2: Servis Odaklı Mimari (ÖNERİLEN)	3
1.3	Mimari Karşılaştırma Tablosu	3
2	İletişim Protokolleri Karşılaştırması	4
2.1	Protokol Performans Grafiği	4
2.2	Protokol Karşılaştırma Tablosu	5
3	Veritabanı Seçenekleri Karşılaştırması	5
3.1	Veritabanı Karşılaştırma Tablosu	7
4	Backend Framework Karşılaştırması	7
4.1	Framework Karşılaştırma Tablosu	9
5	Donanım Seçenekleri Karşılaştırması	9
5.1	Donanım Karşılaştırma Tablosu	11
6	Toplam Maliyet Analizi (3 Senaryo)	12
6.1	Maliyet Senaryoları Özeti	13
7	Sonuç ve Öneriler	13
8	Araştırılması Gereken Konular	14

1 Mimari Yaklaşımların Karşılaştırmalı Analizi

1.1 Mantık 1: Monolitik Mimari

Monolitik Mimari Özellikleri

Açıklama: Tüm bileşenler (Web API, MQTT Listener, Veritabanı) tek bir uygulama içinde çalışır.

Avantajlar:

- Basit geliştirme ve deploy süreci
- Başlangıç maliyeti çok düşük
- Tek kod tabanı, kolay debug
- Düşük network overhead
- Hızlı prototipleme

Dezavantajlar:

- Ölçeklenebilirlik sınırlı
- Tek nokta arızası riski
- Yoğun IoT trafiğinde web arayüzü yavaşlar
- Uzun vadede bakım zorluğu

Maliyet: \$ (En düşük - tek sunucu)

Performans: Orta

1.2 Mantık 2: Servis Odaklı Mimari (ÖNERİLEN)

Servis Odaklı Mimari - ÖNERİLEN

Açıklama: Web uygulaması ve IoT veri işleyici birbirinden bağımsız servisler olarak çalışır.

Avantajlar:

- Her servis bağımsız ölçeklenebilir
- Web arayüzü IoT trafiğinden etkilenmez
- Hata izolasyonu
- Farklı teknolojiler kullanılabilir
- Bakım ve güncelleme kolaylığı

Dezavantajlar:

- Daha karmaşık deployment
- Servisler arası iletişim overhead'i
- İlk kurulum maliyeti daha yüksek

Maliyet: \$\$ (Orta - 2-3 sunucu)

Performans: Yüksek

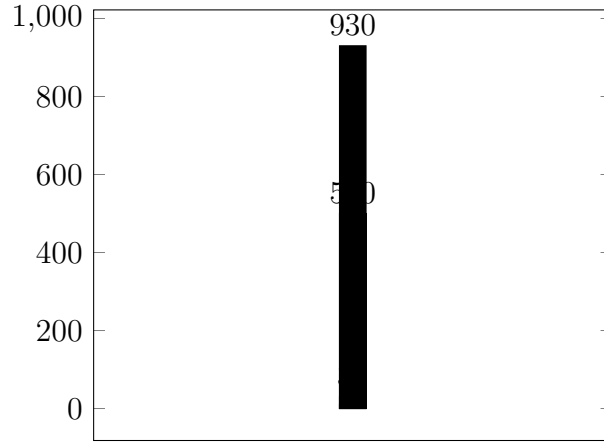
1.3 Mimari Karşılaştırma Tablosu

Kriter	Monolitik	Servis Odaklı	Event-Driven	Serverless
Latency	100-500ms	50-200ms	10-100ms	100-1000ms
Throughput	1K-5K req/s	10K-50K req/s	100K+ req/s	Değişken
Scalability	Düşük	Orta-Yüksek	Çok Yüksek	Otomatik
Cost (10K users)	\$50/ay	\$150/ay	\$500/ay	\$100-300/ay
Complexity	Kolay	Orta	Zor	Orta

Tablo 1: Mimari Yaklaşımların Karşılaştırması

2 İletişim Protokolleri Karşılaştırması

2.1 Protokol Performans Grafiği



Şekil 1: Protokol Performans Karşılaştırması (3G Network)

MQTT Protokolü - ÖNERİLEN

Teknik Özellikler:

- **Header Boyutu:** 2-10 bytes (çok hafif)
- **Bağlantı Tipi:** Persistent (keep-alive)
- **Power Consumption:** Çok Düşük (80µA sleep mode)
- **RAM Kullanımı:** <1KB
- **Battery Life:** 3-5 yıl (2xAA ile)

Performans Avantajları:

- 3G network'te HTTP'den **90 kat daha hızlı**
- Battery impact: %25-30 daha iyi
- IoT telemetry için ideal

2.2 Protokol Karşılaştırma Tablosu

Kriter	MQTT	HTTP/1.1	HTTP/2	WebSocket
Header Size	2-10 bytes	500-1000 bytes	100-300 bytes	10-20 bytes
RAM Usage	<1KB	8KB	4KB	2KB
Power	Çok Düşük	Yüksek	Orta	Orta
3G Throughput	90x HTTP	Baz	2-3x HTTP/1.1	50x HTTP/1.1
Battery Life	3-5 yıl	6-12 ay	8-15 ay	1-2 yıl
IoT Suitability	Mükemmel	Zayıf	Orta	İyi

Tablo 2: İletişim Protokolleri Detaylı Karşılaştırması

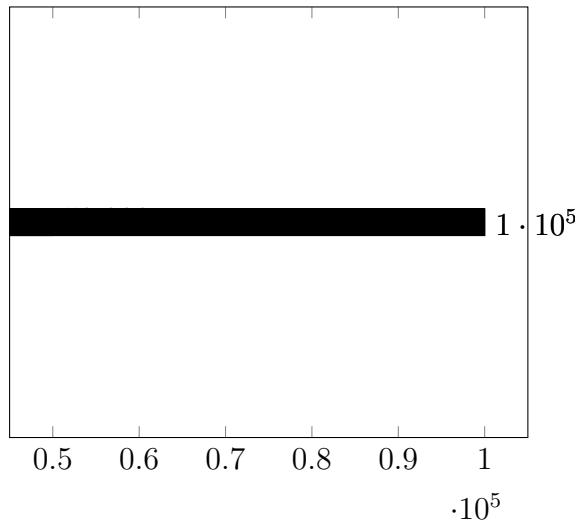
Protokol Seçim Kararı

Karar: MQTT protokolü seçilmiştir.

Gerekçeler:

- IoT için özel olarak tasarlanmış en hafif protokol
- 90 kat daha hızlı throughput (3G network)
- Ultra-düşük güç tüketimi
- Publish/subscribe modeli ile scalable architecture
- Mosquitto broker: Ücretsiz, open-source

3 Veritabanı Seçenekleri Karşılaştırması



Şekil 2: Veritabanı Write Performance Karşılaştırması

PostgreSQL - ÖNERİLEN

Teknik Özellikler:

- **Tip:** Relational (SQL)
- **Write Performance:** 10K-50K writes/sec
- **Read Performance:** 15ms (basit query)
- **Query Language:** SQL (ANSI standard)

Avantajlar:

- Çok geniş ekosistem ve community
- ACID compliance (veri bütünlüğü)
- Django ORM ile mükemmel uyum
- Öğrenme eğrisi kolay (SQL)
- Ücretsiz (open-source)

InfluxDB

Teknik Özellikler:

- **Tip:** Time-Series NoSQL
- **Write Performance:** 100K+ writes/sec
- **Read Performance:** 29ms
- **Compression:** Yüksek (90%+)

Avantajlar: IoT için özel optimizasyon, çok yüksek write throughput

Dezavantajlar: Django ORM ile doğrudan entegrasyon yok

TimescaleDB

Teknik Özellikler:

- **Tip:** Time-Series (PostgreSQL extension)
- **Write Performance:** 50K-100K writes/sec
- **Read Performance:** 20-25ms
- **Query Language:** SQL (PostgreSQL compatible)

Avantajlar: PostgreSQL ekosistemi ile tam uyumluluk, SQL kullanımı

3.1 Veritabanı Karşılaştırma Tablosu

Özellik	PostgreSQL	InfluxDB	TimescaleDB
Type	Relational	Time-Series NoSQL	Time-Series (PG)
Write Perf.	10K-50K/s	100K+/s	50K-100K/s
Read Perf.	15ms	29ms	20-25ms
Query Lang.	SQL	InfluxQL/Flux	SQL
Django ORM	Mükemmel	Yok	İyi
Cost	Ücretsiz	Ücretsiz/Enterprise	Ücretsiz

Tablo 3: Veritabanı Detaylı Karşılaştırması

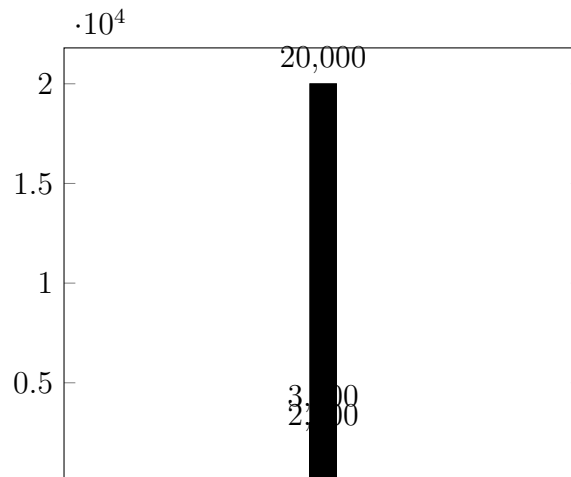
Veritabanı Seçim Kararı

Karar: PostgreSQL (başlangıç), ileride TimescaleDB extension

Gerekçeler:

- Django ORM ile mükemmel uyum (geliştirme hızı)
- SQL bilgisi (öğrenme kolaylığı)
- Başlangıç/orta ölçek için yeterli performans
- İleride TimescaleDB extension ile zaman serisi optimizasyonu

4 Backend Framework Karşılaştırması



Şekil 3: Backend Framework Performance Benchmark

Django - ÖNERİLEN (Başlangıç)

Teknik Özellikler:

- **Performance:** Orta - 2,000 req/s
- **Async Support:** Sınırlı (Django 3.1+)
- **Learning Curve:** Kolay
- **Built-in Features:** Çok fazla (ORM, Admin, Auth)

Avantajlar:

- Hızlı geliştirme (rapid development)
- Admin panel hazır (zaman tasarrufu)
- Güçlü ORM (database abstraction)
- Built-in authentication ve security
- Çok büyük community

FastAPI - Yüksek Performans

Teknik Özellikler:

- **Performance:** Çok Yüksek - 15,000-20,000 req/s
- **Async Support:** Native (async-first)
- **Learning Curve:** Orta
- **API Documentation:** Otomatik (OpenAPI/Swagger)

Avantajlar: Çok yüksek performans, async/await native support, IoT telemetry için ideal

Flask

Teknik Özellikler:

- **Performance:** İyi - 2,000-3,000 req/s
- **Learning Curve:** Çok Kolay
- **Flexibility:** Çok yüksek

Avantajlar: Çok basit ve hafif, öğrenmesi çok kolay

Dezavantajlar: Çok az built-in feature, büyük projelerde yönetim zor

4.1 Framework Karşılaştırma Tablosu

Kriter	Django	FastAPI	Flask
Performance	Orta	Çok Yüksek	İyi
Throughput	2K req/s	15-20K req/s	2-3K req/s
Learning	Kolay	Orta	Çok Kolay
Features	Çok fazla	Minimal	Minimal
Async	Sınırlı	Native	Sınırlı
Admin Panel	Hazır	Yok	Yok
IoT Suitability	İyi	Mükemmel	Orta

Tablo 4: Backend Framework Detaylı Karşılaştırması

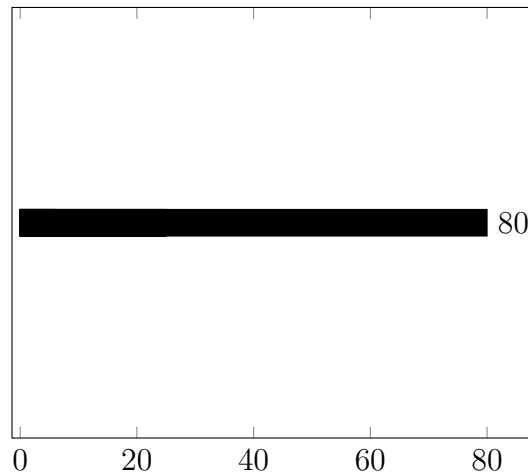
Framework Seçim Kararı

Karar: Django (başlangıç), kritik endpoints için **FastAPI** (hibrit)

Gerekçeler:

- Django: Hızlı geliştirme, admin panel hazır, ORM mükemmel
- FastAPI: Yüksek performans gereken IoT API endpoints için
- 1. sınıf öğrencisi için Django öğrenme kolaylığı

5 Donanım Seçenekleri Karşılaştırması



Şekil 4: Donanım Maliyet Karşılaştırması

ESP32 - ÖNERİLEN

Teknik Özellikler:

- **Fiyat:** \$3-6 (tekil)
- **CPU:** Dual-core 240MHz
- **RAM:** 520KB
- **WiFi/Bluetooth:** Dahili (802.11 b/g/n, BT 4.2)
- **Power:** 80µA sleep mode
- **Battery Life:** 3-5 yıl (2xAA ile)

Avantajlar:

- **En iyi fiyat/performans**
- WiFi + Bluetooth dahili (ek shield gerekmez)
- Ultra-düşük güç tüketimi
- IoT için ideal
- Arduino IDE ile programlanabilir

Raspberry Pi 4/5

Teknik Özellikler:

- **Fiyat:** \$60-80
- **CPU:** Quad-core 1.5-2.4GHz (ARM Cortex)
- **RAM:** 2-8GB
- **OS:** Raspberry Pi OS (Linux)
- **Power:** 500mA-3A (yüksek)

Avantajlar: Tam bilgisayar (Linux), çok güçlü (edge computing)

Dezavantajlar: Pahalı (ESP32'den 10-15x), yüksek güç tüketimi

Arduino Uno

Teknik Özellikler:

- **Fiyat:** \$18-25 (orijinal)
- **CPU:** 16MHz AVR (ATmega328P)
- **RAM:** 2KB
- **WiFi/Bluetooth:** Yok (shield gerekli)

Avantajlar: Çok kolay öğrenim, geniş community

Dezavantajlar: WiFi yok, ESP32'den daha az yetenekli

5.1 Donanım Karşılaştırma Tablosu

Özellik	ESP32	Raspberry Pi 4	Arduino Uno
Fiyat	\$3-6	\$60-80	\$18-25
CPU	240MHz Dual-core	1.5-2.4GHz Quad-core	16MHz
RAM	520KB	2-8GB	2KB
WiFi/BT	Dahili	Ethernet/WiFi model	Yok (shield)
Power	80µA sleep	500mA-3A	Düşük
Battery	3-5 yıl	Saatler-günler	Uzun
Use Case	IoT sensors	Gateway, edge computing	Eğitim
Fiyat/Perf.	En iyi	Pahalı	Orta

Tablo 5: Donanım Detaylı Karşılaştırması

Maliyet Analizi (10 Sensör + 1 Gateway)

ESP32 Bazlı Sistem:

$10 \times \text{ESP32} (\$5) + 1 \times \text{Raspberry Pi} (\$80) = \$130$

Raspberry Pi Bazlı Sistem:

$10 \times \text{Raspberry Pi} (\$80) + 1 \times \text{Raspberry Pi} (\$80) = \$880$

Tasarruf: \$750 (%85 daha ucuz!)

Donanım Seçim Kararı

Karar: ESP32 (sensör node'ları) + Raspberry Pi (gateway)

Gerekçeler:

- ESP32: En iyi fiyat/performans oranı
- WiFi + Bluetooth dahili (ek maliyet yok)
- Ultra-düşük güç tüketimi (battery-powered sensors)
- 10 sensör için \$130 vs \$880 (%85 tasarruf)

6 Toplam Maliyet Analizi (3 Senaryo)

Senaryo 1: Budget-Friendly (Öğrenci Projesi)

Mimari: Monolitik (Mantık 1)

Donanım: 3× ESP32 (\$15)

Yazılım: Django + PostgreSQL + MQTT (Mosquitto)

Hosting: Local/Raspberry Pi (\$0-80)

Toplam Maliyet: \$15-95 (one-time)

Artılar: Çok ucuz, basit, hızlı başlangıç

Senaryo 2: Balanced - ÖNERİLEN

Mimari: Servis Odaklı (Mantık 2)

Donanım: 10× ESP32 (\$50) + 1× Raspberry Pi (\$80)

Yazılım: Django (web) + FastAPI (IoT API) + PostgreSQL + MQTT

Hosting: Cloud VPS (\$10-20/ay) veya local

Toplam Maliyet: \$130-150 (one-time) + \$10-20/ay (cloud)

Artılar: Ölçeklenebilir, profesyonel, production-ready

Senaryo 3: Enterprise-Grade

Mimari: Event-Driven (Mantık 3)

Donanım: 50× ESP32 (\$250) + 2× Raspberry Pi (\$160)

Yazılım: FastAPI + Kafka + InfluxDB + Docker + Kubernetes

Hosting: Cloud (AWS/Azure)

Toplam Maliyet: \$410+ (one-time) + \$100-500/ay (cloud)

Artılar: Yüksek ölçeklenebilirlik, enterprise-level

6.1 Maliyet Senaryoları Özeti

Kriter	Budget	Balanced	Enterprise
Hardware	\$15	\$130	\$410
Software	Ücretsiz	Ücretsiz	Ücretsiz (OSS)
Cloud (aylık)	\$0	\$10-20	\$100-500
Toplam (ilk yıl)	\$15-95	\$250-390	\$1610-6410
Sensors	3	10	50
Complexity	Düşük	Orta	Yüksek
Scalability	Yok	İyi	Mükemmel

Tablo 6: Maliyet Senaryoları Karşılaştırması

7 Sonuç ve Öneriler

Senin Projen İçin Nihai Öneriler (1. Sınıf Öğrencisi)

1. Mimari: Servis Odaklı (Mantık 2)

- Django (web panel + admin)
- Ayır Python script (MQTT consumer)
- PostgreSQL (veri deposu)

2. İletişim: MQTT (kesinlikle)

- Mosquitto broker (ücretsiz)
- Paho MQTT client (Python)
- 90x faster than HTTP

3. Veritabanı: PostgreSQL

- İleride TimescaleDB extension eklenebilir
- Django ORM ile mükemmel uyum

4. Backend: Django (şimdilik)

- Admin panel hazır
- ORM sayesinde hızlı geliştirme

5. Donanım: ESP32

- 3-5 adet ile başla (\$15-25)
- WiFi dahili, güç tüketimi düşük

Toplam Başlangıç Maliyeti: \$50-100

8 Araştırılması Gereken Konular

Önümüzdeki Haftalar İçin Araştırma Konuları

1. MQTT Protocol Deep Dive:

- QoS seviyeleri (0, 1, 2)
- Topic tasarımı
- Last Will & Testament
- Retained messages

2. PostgreSQL Optimization:

- Indexing strategies
- Partitioning (zaman serisi için)
- Connection pooling (PgBouncer)

3. Django Best Practices:

- Class-Based Views vs Function-Based Views
- Django ORM optimization
- Caching (Redis)

4. ESP32 Programming:

- Arduino IDE kurulumu
- WiFi bağlantısı
- Deep sleep modes (güç tasarrufu)
- MQTT publishing